DATA UNDERSTANDING AND PREPROCESSING

Step-by-step practical lab using the Diabetes dataset

Course topic: Data Understanding and Data Preprocessing

Programming language: Python | Main library: Pandas

Dataset: diabetes.csv (768 records, 9 columns)

STUDENT PRACTICAL WORKBOOK

This workbook guides you from opening a raw CSV file to inspecting, cleaning, preparing, and exporting data. Run every code cell in order and compare your output with the checkpoints.

1. Lab overview

In this practical lab, you will work with a healthcare-related dataset commonly known as the Pima Indians Diabetes dataset. Each row represents one record and each column stores one variable. The task is not to diagnose a patient. The goal is to learn how to inspect data quality and prepare a dataset for later analysis or machine learning.

Learning outcomes

- Load a CSV file into a Pandas DataFrame.
- Understand rows, columns, features, and the target variable.
- Inspect data types, summary statistics, class distribution, and data quality.
- Identify zero values that may be placeholders for missing medical measurements.
- Handle missing values and review possible outliers.
- Split data correctly and apply imputation and scaling without data leakage.
- Export cleaned and processed data files.

Required software

- Python 3.10 or later (recommended), Jupyter Notebook / JupyterLab / Google Colab, and the libraries pandas, numpy, matplotlib, seaborn, and scikit-learn.

PYTHON CODE

```
pip install pandas numpy matplotlib seaborn scikit-learn
```

If you use Google Colab, these libraries are usually pre-installed. You may need to upload diabetes.csv to the Colab session.

Folder structure

PYTHON CODE

```
data-preprocessing-lab/
    diabetes.csv
    diabetes_cleaned.csv
    diabetes_train.csv
    diabetes_test.csv
    diabetes_processed.csv
    lab.ipynb
```

Keep the original diabetes.csv unchanged. Save all new files separately so that you can always return to the raw data.

2. Understand the problem and the dataset

Problem statement: A data analyst receives a raw dataset containing medical measurements and a binary outcome. Before analyzing the data or building a predictive model, the analyst must understand what each field means, inspect the quality of the records, identify suspicious values, and prepare a reliable version of the data.

Dataset structure

Item	Description
Number of records	768 rows
Number of columns	9 columns
Data format	CSV (comma-separated values)
Domain	Healthcare / diabetes research dataset
Target column	Outcome
Task type	Binary classification (0 or 1)

Meaning of each column

Column	Meaning	Role / unit
Pregnancies	Number of times pregnant	Feature; count
Glucose	Plasma glucose concentration	Feature; mg/dL in this dataset
BloodPressure	Diastolic blood pressure	Feature; mm Hg
SkinThickness	Triceps skin-fold thickness	Feature; mm
Insulin	2-hour serum insulin	Feature; mu U/mL
BMI	Body Mass Index	Feature; kg/m²
DiabetesPedigreeFunction	A score related to family-history influence	Feature; score
Age	Age of the patient	Feature; years
Outcome	0 = non-diabetic; 1 = diabetic	Target; class label

The eight input columns are called features (X). Outcome is the target (y) that a classification model may later try to predict.

Important: This dataset is for educational practice. It is not a clinical diagnostic tool, and model predictions must not be used to make medical decisions.

3. Step 1 — Set up the notebook and import libraries

Create a new notebook named lab.ipynb. Run the following cell first.

PYTHON CODE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)

print("Libraries imported successfully!")
```

Expected result / checkpoint
Libraries imported successfully!

Why these libraries? Pandas handles tabular data; NumPy supports numerical operations; Matplotlib and Seaborn create charts; scikit-learn provides splitting, imputation, scaling, and machine-learning utilities.

4. Step 2 — Load the CSV file

Make sure diabetes.csv is in the same folder as your notebook. If it is stored in another folder, provide the correct path.

PYTHON CODE

```
df = pd.read_csv("diabetes.csv")

print("Dataset loaded successfully!")
print(df.head())
```

Expected result / checkpoint
Dataset loaded successfully!
Pregnancies Glucose BloodPressure SkinThickness Insulin BMI DiabetesPedigreeFunction Age Outcome
0 6 148 72 35 0 33.6 0.627 50 1
1 1 85 66 29 0 26.6 0.351 31 0
2 8 183 64 0 0 23.3 0.672 32 1
3 1 89 66 23 94 28.1 0.167 21 0
4 0 137 40 35 168 43.1 2.288 33 1

If Python raises FileNotFoundError, check the filename, folder, and working directory. The filename must match exactly, including the .csv extension.

5. Step 3 — Inspect the first and last records

PYTHON CODE

```
print("First 5 rows:")
print(df.head())

print("\\nLast 5 rows:")
print(df.tail())

print("\\nA random sample of 5 rows:")
print(df.sample(5, random_state=42))
```

head() shows the first records, tail() shows the last records, and sample() selects random records. A fixed random_state makes the sample reproducible.

Expected result / checkpoint
Checkpoint: head() should show records with index 0 to 4. The first record has Pregnancies=6, Glucose=148, and Outcome=1.

6. Step 4 — Check dataset dimensions and column names

PYTHON CODE

```
print("Shape:", df.shape)
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])

print("\\nColumn names:")
print(df.columns.tolist())
```

Expected result / checkpoint
Shape: (768, 9)
Number of rows: 768
Number of columns: 9

Column names:
['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome']

The shape attribute returns (number_of_rows, number_of_columns). A row is one observation; a column is one variable.

7. Step 5 — Inspect data types and non-null counts

PYTHON CODE

```
df.info()
```

Expected result / checkpoint
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
Column Non-Null Count Dtype
0 Pregnancies 768 non-null int64
1 Glucose 768 non-null int64
2 BloodPressure 768 non-null int64
3 SkinThickness 768 non-null int64
4 Insulin 768 non-null int64
5 BMI 768 non-null float64
6 DiabetesPedigreeFunction 768 non-null float64
7 Age 768 non-null int64
8 Outcome 768 non-null int64

The integer columns are stored as int64. BMI and DiabetesPedigreeFunction are stored as float64. Non-null count tells you how many values are present in each column.

8. Step 6 — Calculate descriptive statistics

PYTHON CODE

```
print(df.describe().round(2))
```

Expected result / checkpoint
Key values (rounded):
Pregnancies: mean 3.85, min 0, max 17
Glucose: mean 120.89, min 0, max 199
BloodPressure: mean 69.11, min 0, max 122
SkinThickness: mean 20.54, min 0, max 99
Insulin: mean 79.80, min 0, max 846
BMI: mean 31.99, min 0.0, max 67.1
Age: mean 33.24, min 21, max 81
Outcome: mean 0.35, min 0, max 1

describe() reports count, mean, standard deviation, minimum, quartiles, and maximum. A minimum of zero in some medical measurement columns is a warning that needs investigation. Do not assume that every zero is wrong.

9. Step 7 — Understand the target variable

PYTHON CODE

```
print(df["Outcome"].value_counts().sort_index())
print("\\nPercentage:")
print((df["Outcome"].value_counts(normalize=True).sort_index() * 100).round(1))
sns.countplot(data=df, x="Outcome")
plt.title("Distribution of Outcome")
plt.xlabel("Outcome (0 = non-diabetic, 1 = diabetic)")
plt.ylabel("Number of records")
plt.show()
```

Expected result / checkpoint
Outcome
0 500
1 268

Percentage:

0 65.1
1 34.9

There are 500 records with Outcome=0 and 268 records with Outcome=1. The classes are not perfectly balanced. This is worth remembering when evaluating a future classification model.

10. Step 8 — Check missing values

PYTHON CODE

```
missing_count = df.isnull().sum()
missing_percent = (df.isnull().mean() * 100).round(2)

missing_report = pd.DataFrame({
    "Missing_Count": missing_count,
    "Missing_Percent": missing_percent
})

print(missing_report)
```

Expected result / checkpoint
All nine columns have Missing_Count = 0 and Missing_Percent = 0.00%.

There are no explicit NaN values in the original CSV. However, a dataset can contain hidden missing values encoded as special numbers such as 0, -1, or 999. We must understand the meaning of each field before deciding what is missing.

11. Step 9 — Investigate zero values

In this dataset, zero is valid for Pregnancies (a person may have had zero pregnancies) and Outcome (class 0 is meaningful). However, zero measurements for Glucose, BloodPressure, SkinThickness, Insulin, and BMI may represent measurements that were not recorded.

PYTHON CODE

```
zero_columns = [
    "Glucose", "BloodPressure", "SkinThickness",
    "Insulin", "BMI"
]

print(df[zero_columns].eq(0).sum())
```

Expected result / checkpoint
Glucose 5
BloodPressure 35
SkinThickness 227
Insulin 374
BMI 11

The count of zero values in the selected columns is 652 in total (counted per column; a record may contain more than one zero). Insulin has the largest number of zeros.

Do not replace zeros in Pregnancies or Outcome. Also do not automatically replace every zero in every column. The decision must be based on the meaning of the variable and the dataset documentation.

12. Step 10 — Check duplicate records

PYTHON CODE

```
duplicate_count = df.duplicated().sum()
print("Number of duplicate rows:", duplicate_count)
```

Expected result / checkpoint
Number of duplicate rows: 0

There are no exact duplicate rows in the supplied dataset. If duplicates existed, investigate them before removing them. Two patients can legitimately have identical measured values, so duplication should not be judged only by one or two columns.

13. Step 11 — Create a data quality summary

PYTHON CODE

```
quality_report = pd.DataFrame({
    "Data_Type": df.dtypes.astype(str),
    "Missing_Values": df.isnull().sum(),
    "Zero_Values": (df == 0).sum(),
    "Unique_Values": df.nunique()
})

print(quality_report)
```

Use the report to summarize what you found: no explicit missing values, no exact duplicate rows, but suspicious zeros in selected medical measurement fields. The next step is to create a cleaning plan.

14. Step 12 — Convert suspicious zeros to missing values

For the five selected measurement columns only, replace 0 with NaN. Keep the original DataFrame unchanged by creating a copy.

PYTHON CODE

```
df_clean = df.copy()

measurement_columns = [
    "Glucose", "BloodPressure", "SkinThickness",
    "Insulin", "BMI"
]

df_clean[measurement_columns] = (
    df_clean[measurement_columns].replace(0, np.nan)
)

print(df_clean[measurement_columns].isnull().sum())
```

Expected result / checkpoint
Glucose 5
BloodPressure 35
SkinThickness 227
Insulin 374
BMI 11

Why use np.nan? Pandas and scikit-learn recognize NaN as a missing value. This makes it possible to count and impute missing measurements using standard tools.

15. Step 13 — Fill missing values with the median (exploratory cleaning)

For a simple data-cleaning exercise, calculate the median of each measurement column and fill its missing values. The median is less affected by extreme values than the mean.

PYTHON CODE

```
for col in measurement_columns:
    median_value = df_clean[col].median()
    df_clean[col] = df_clean[col].fillna(median_value)
    print(col, "median =", round(median_value, 2))

print("\\nMissing values after filling:")
print(df_clean[measurement_columns].isnull().sum())
```

Expected result / checkpoint
Median values (rounded):
Glucose = 117.00
BloodPressure = 72.00
SkinThickness = 29.00
Insulin = 125.00

```
BMI = 32.00

Missing values after filling:
All five columns = 0
```

After this step, df_clean has no missing values in the five selected columns. This version is useful for demonstrating a simple cleaning operation. For a machine-learning workflow, use the train/test-safe pipeline in Steps 19–21 instead of calculating medians on the complete dataset.

16. Step 14 — Review possible outliers with boxplots

PYTHON CODE

```
plt.figure(figsize=(12, 6))
sns.boxplot(data=df_clean[measurement_columns])
plt.title("Boxplots of Medical Measurement Features")
plt.xticks(rotation=25)
plt.tight_layout()
plt.show()
```

A boxplot highlights the median, quartiles, and points beyond the whiskers. These points are potential outliers, not automatically errors. For example, a high insulin measurement may be unusual but still possible.

Do not delete outliers without a reason. Investigate the record, domain meaning, and the effect on the analysis before changing it.

17. Step 15 — Compare distributions before and after cleaning

PYTHON CODE

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

sns.histplot(df["Glucose"], bins=25, kde=True, ax=axes[0])
axes[0].set_title("Glucose: Original Data")

sns.histplot(df_clean["Glucose"], bins=25, kde=True, ax=axes[1])
axes[1].set_title("Glucose: After Zero Handling")

plt.tight_layout()
plt.show()
```

Compare the shape of the distributions. Replacing zero placeholders with median values changes the distribution. This is a deliberate assumption for practice and should be documented.

18. Step 16 — Save the exploratory cleaned dataset

PYTHON CODE

```
df_clean.to_csv("diabetes_cleaned.csv", index=False)

print("Cleaned dataset exported successfully!")
print("New shape:", df_clean.shape)
print("Missing values:", df_clean.isnull().sum().sum())
```

```
Expected result / checkpoint
Cleaned dataset exported successfully!
New shape: (768, 9)
Missing values: 0
```

index=False prevents Pandas from writing the DataFrame index as an extra column. Open diabetes_cleaned.csv in a spreadsheet application to verify the exported file.

19. Step 17 — Separate features (X) and target (y)

For classification, Outcome is the target. All other columns are input features.

PYTHON CODE

```
X = df[[
    "Pregnancies", "Glucose", "BloodPressure",
    "SkinThickness", "Insulin", "BMI",
    "DiabetesPedigreeFunction", "Age"
]].copy()

y = df["Outcome"].copy()

# Convert only the selected zero placeholders to NaN in X
X[measurement_columns] = X[measurement_columns].replace(0, np.nan)

print("X shape:", X.shape)
print("y shape:", y.shape)
print("\\nTarget distribution:")
print(y.value_counts().sort_index())
```

Expected result / checkpoint
X shape: (768, 8)
y shape: (768,)

0 500
1 268

Notice that Outcome is not included in X. Including the target in the feature matrix would cause target leakage.

20. Step 18 — Split into training and testing sets

Split the data before fitting an imputer or scaler. Use stratify=y to keep approximately the same class proportions in both sets.

PYTHON CODE

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("X_train:", X_train.shape)
print("X_test :", X_test.shape)
print("y_train:", y_train.shape)
print("y_test :", y_test.shape)
```

Expected result / checkpoint
X_train: (614, 8)
X_test : (154, 8)
y_train: (614,)
y_test : (154,)

The 80/20 split produces 614 training records and 154 testing records. random_state=42 makes the split reproducible. stratify=y preserves the class ratio as closely as possible.

21. Step 19 — Build a preprocessing pipeline

The pipeline first fills missing measurements using the median learned from the training set. It then standardizes numeric features using the training-set mean and standard deviation. The same learned transformations are applied to the test set.

PYTHON CODE

```
preprocessor = Pipeline(steps=[
```

```
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

print("Processed training shape:", X_train_processed.shape)
print("Processed testing shape :", X_test_processed.shape)
print("Any NaN in training data:",
      np.isnan(X_train_processed).sum())
print("Any NaN in testing data:",
      np.isnan(X_test_processed).sum())
```

Expected result / checkpoint
Processed training shape: (614, 8)
Processed testing shape : (154, 8)
Any NaN in training data: 0
Any NaN in testing data: 0

Key concept — data leakage: Do not fit the imputer or scaler on the full dataset before splitting. Doing so allows information from the test set to influence the preprocessing parameters. fit_transform() is used on training data; transform() is used on test data.

22. Step 20 — Check the scaling result

PYTHON CODE

```
processed_train_df = pd.DataFrame(
    X_train_processed,
    columns=X_train.columns
)

print(processed_train_df.describe().loc[["mean", "std"]].round(2))
```

After StandardScaler, each feature in the training set should have a mean close to 0 and a standard deviation close to 1. Small differences may appear due to rounding.

Expected result / checkpoint
Expected pattern: training feature means are approximately 0.00 and standard deviations are approximately 1.00.

23. Step 21 — Export train and test data

Export the processed feature matrices and their labels. This example keeps the target column separate from the scaled features.

PYTHON CODE

```
train_export = processed_train_df.copy()
train_export["Outcome"] = y_train.reset_index(drop=True)

test_export = pd.DataFrame(
    X_test_processed,
    columns=X_test.columns
)
test_export["Outcome"] = y_test.reset_index(drop=True)

train_export.to_csv("diabetes_train.csv", index=False)
test_export.to_csv("diabetes_test.csv", index=False)

print("Training and testing files exported.")
print("Train file shape:", train_export.shape)
print("Test file shape :", test_export.shape)
```

Expected result / checkpoint
Training and testing files exported.
Train file shape: (614, 9)
Test file shape : (154, 9)

These files contain scaled features and the original Outcome labels. Keep the fitted preprocessing pipeline if you later need to transform new records using exactly the same medians and scaling parameters.

24. Step 22 — Final verification checklist

- The original diabetes.csv file was not overwritten.
- The dataset contains 768 rows and 9 columns.
- Outcome is treated as the target variable.
- Zero values were investigated column by column.
- Zeros were converted to missing values only in the selected measurement columns.
- No explicit missing values remain after preprocessing.
- The data was split before fitting the imputer and scaler.
- The training set contains 614 rows and the testing set contains 154 rows.
- The exported CSV files can be opened and checked.

25. Student tasks

Complete these tasks independently. Include code, output screenshots, and short written explanations in your submission.

- Print the mean, median, minimum, and maximum of Glucose, BMI, and Age.
- Create a bar chart showing the number of records for each Outcome class.
- Find the five records with the highest Insulin values. Display only Insulin, Glucose, BMI, Age, and Outcome.
- Calculate the percentage of zero values in each of the five selected measurement columns.
- Compare mean imputation and median imputation for Insulin. Which method is more appropriate for a skewed distribution, and why?
- Create a boxplot for BMI and explain what a potential outlier means.
- Explain why Pregnancies=0 and Outcome=0 must not be treated as missing values.
- Explain in your own words why the train/test split must happen before fitting the imputer and scaler.

Submission requirements

- Submit the completed notebook (.ipynb) and the generated CSV files.
- Add a short report (1–2 pages) describing the data quality issues and the decisions made during preprocessing.
- Every chart must have a title and labeled axes.
- Do not submit only screenshots; include the code cells and their outputs.

26. Troubleshooting

Problem	Possible solution
FileNotFoundError	Check the current working directory and CSV filename.
ModuleNotFoundError	Install the missing library with pip install package_name.
KeyError: column name	Print df.columns and copy the exact column name.
Output looks different	Check whether the original CSV was changed and whether the same code order and random_state were used.
A chart does not appear	Run the cell again and check that plt.show() is included.
NaN remains after preprocessing	Check that the selected columns were converted and that the imputer was fitted on training data.

27. Key concepts to remember

Concept	Meaning
Data understanding	Learning what the dataset, rows, columns, and values represent.
Data quality	Checking missing values, invalid values, duplicates, and suspicious records.
Data preprocessing	Transforming raw data into a form suitable for analysis or modeling.
Imputation	Replacing missing values with a selected estimate such as the median.
Scaling	Putting numeric features on comparable scales.
Train/test split	Separating data for model training and independent evaluation.
Data leakage	Using information from outside the training data when fitting preprocessing or a model.

Final message: Data preprocessing is not simply deleting or replacing values. Every change should have a clear reason, be documented, and preserve the meaning of the data as much as possible.

Using Visual Studio Code (VS Code)

This section replaces the Jupyter Notebook setup instructions. Students will complete the practical lab using a normal Python script (.py) in VS Code.

Recommended project folder

```
diabetes-preprocessing-lab/
    diabetes.csv
    lab.py
```

Keep diabetes.csv and lab.py in the same folder. Do not rename the CSV file unless you also update the path in the Python code.

1. Install the required software

Install Python 3.10 or later and Visual Studio Code. In VS Code, open Extensions (Ctrl+Shift+X / Cmd+Shift+X), search for Python by Microsoft, and install it. The Python extension provides interpreter selection, running, and debugging support.

2. Open the project folder

In VS Code, choose File → Open Folder and select diabetes-preprocessing-lab. Do not open only lab.py; opening the folder makes it easier for Python to find diabetes.csv.

3. Open the VS Code terminal

Choose Terminal → New Terminal. Run the commands below in the terminal. On Windows, use PowerShell; on macOS/Linux, use the integrated terminal.

Windows (PowerShell)

```
py -m venv .venv
.venv\\Scripts\\Activate.ps1
python -m pip install --upgrade pip
python -m pip install pandas numpy matplotlib seaborn scikit-learn
```

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install pandas numpy matplotlib seaborn scikit-learn
```

If PowerShell blocks activation, students can use the interpreter without activating the environment, or follow their institution's approved PowerShell policy. Do not ask students to change system-wide security settings unnecessarily.

4. Select the Python interpreter

Press Ctrl+Shift+P (Windows/Linux) or Cmd+Shift+P (macOS), search for Python: Select Interpreter, and select the interpreter inside the project's .venv folder. The selected environment should be the one where the libraries were installed.

5. Create lab.py and test the environment

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

print("Python environment is ready!")
print("Pandas version:", pd.__version__)
```

Save the file, then run it using the Run Python File button at the top-right of the editor, or run this command in the terminal:

```
python lab.py
```

Expected output

```
Python environment is ready!
Pandas version: (version number depends on installation)
```

6. Run the practical lab in VS Code

Copy the code examples from the practical guide into lab.py in the same order, from importing libraries through exporting the files. Add print() around values you want to inspect. Save the file before running it.

For a beginner-friendly class, ask students to run the whole script after completing each section. Alternatively, they can temporarily comment out later sections while testing an earlier section. They should not run later steps before creating the variables those steps depend on.

7. Working with charts

When the script reaches plt.show(), Matplotlib opens a chart window or displays the chart through the VS Code Python environment. Close the chart window to allow the script to continue if execution pauses. If a chart does not appear, check that plt.show() is present and that the correct Python interpreter is selected.

8. Verify the output files

After the script completes, the Explorer panel should show the generated CSV files in the project folder, for example diabetes_cleaned.csv, diabetes_train.csv, and diabetes_test.csv. These files are created in the current working directory. Keep the original diabetes.csv unchanged.

9. Common VS Code problems

Problem	What to do
No module named pandas	Select the .venv interpreter and install packages in that environment.
FileNotFoundError: diabetes.csv	Open the project folder, check the CSV filename, and run from that folder.
python command not found	On Windows try py; on macOS/Linux try python3. Check that Python is installed and available in PATH.
Chart window seems to pause execution	Close the chart window after inspecting it; the script may continue afterwards.
Wrong interpreter selected	Use Python: Select Interpreter and choose the project's .venv interpreter.

10. Student completion checklist

- I opened the entire project folder in VS Code.
- I selected the correct Python interpreter (.venv).
- I installed pandas, numpy, matplotlib, seaborn, and scikit-learn.
- I placed diabetes.csv beside lab.py.
- I ran the code in the correct order and checked the outputs.
- I generated and checked the required CSV output files.

Instructor note: The original practical guide's code checkpoints remain the expected results. The main change is the development environment: students use lab.py and the VS Code terminal instead of a Jupyter Notebook.